Language Modeling from Scratch

A Comprehensive Guide to Building Large Language Models

Based on Stanford CS336

CS336 Textbook Project

April 3, 2026

Preface

This book is a comprehensive companion to Stanford's CS336: Language Modeling from Scratch. It follows the course's "from scratch" philosophy—building every component of a modern language model, from tokenization to alignment, with full mathematical rigor and detailed implementation guidance.

Who This Book Is For

You are a computer science undergraduate (junior or senior level) who:

- Can program fluently in Python
- Has taken introductory courses in machine learning and linear algebra
- May have used AI tools like ChatGPT or GitHub Copilot
- Does *not* necessarily know NLP, linguistics, CUDA, or distributed systems

If some of those prerequisites feel shaky, Part 0 provides self-contained refreshers on everything you need.

How to Read This Book

The book is organized into six parts plus appendices:

Part 0: Prerequisites Probability, linear algebra, PyTorch, GPU basics, and linguistics background

Part I: Foundations The language modeling problem, tokenization, and the path from n-grams to neural models

Part II: Transformer Internals Attention, Transformer blocks, normalization, and architectural variants

Part III: Systems Resource accounting, GPUs, kernels, parallelism, and inference

Part IV: Scaling and Evaluation Scaling laws, model recipes, and benchmarking

Part V: Data Data sources, filtering, deduplication, and mixing

Part VI: Alignment Instruction tuning, RLHF, DPO, reasoning RL, and safety

You can read sequentially, or follow one of the suggested reading paths in Chapter 0.

Neuroscience Analogies

Throughout this book, we use analogies from neuroscience and brain science to build intuition. Every analogy box clearly states where the analogy helps, where it breaks down, and what intuition you should keep. We never imply that language models are brains—but the parallels can sharpen your understanding of both.

Acknowledgments

This book builds on the materials, lectures, and assignments created by Tatsunori Hashimoto, Percy Liang, and the CS336 teaching staff. We are grateful for their vision of teaching language modeling from scratch.

Contents

Preface **iii**

I Orientation and Prerequisites **1**

1 How to Use This Book **3**
1.1 The "From Scratch" Philosophy . . . 3
1.2 How to Read This Book . . . 3
1.2.1 Sequential Reading . . . 3
1.2.2 "I Know ML" Path . . . 4
1.2.3 Systems Focus . . . 4
1.2.4 Assignment Companion . . . 4
1.3 Notation Conventions . . . 4
1.4 Neuroscience Analogies . . . 4
1.5 Notation Table . . . 5

2 Probability, Entropy, and Information Theory **7**
2.1 Big Picture . . . 7
2.2 Probability Distributions over Tokens . . . 8
2.3 Entropy: Measuring Uncertainty . . . 8
2.3.1 Definition and Intuition . . . 8
2.3.2 Key Properties . . . 9
2.3.3 Worked Example: Computing Entropy . . . 9
2.4 Cross-Entropy: Measuring Model Quality . . . 10
2.4.1 Definition . . . 10
2.4.2 Cross-Entropy as a Training Loss . . . 10
2.5 KL Divergence: The Gap Between Distributions . . . 11
2.6 Maximum Likelihood and Cross-Entropy . . . 12
2.7 Perplexity: The Standard Evaluation Metric . . . 12
2.7.1 Worked Example: Perplexity Calculation . . . 13

2.8 Shannon's Game and the Entropy Rate of English . . . 14
2.9 Common Misconceptions . . . 14
2.10 Exercises . . . 15
2.11 Further Reading . . . 16

3 Linear Algebra and Matrix Calculus 17
3.1 Big Picture . . . 17
3.2 Formal Setup and Notation . . . 17
3.3 Main Ideas . . . 17
3.4 Worked Example . . . 17
3.5 Systems and Implementation Consequences . . . 17
3.6 Common Misconceptions . . . 18
3.7 Exercises . . . 18
3.8 Further Reading . . . 18

4 PyTorch and Tensor Computation 19
4.1 Big Picture . . . 19
4.2 Formal Setup and Notation . . . 19
4.3 Main Ideas . . . 19
4.4 Worked Example . . . 19
4.5 Systems and Implementation Consequences . . . 19
4.6 Common Misconceptions . . . 20
4.7 Exercises . . . 20
4.8 Further Reading . . . 20

5 CUDA and GPU Execution 21
5.1 Big Picture . . . 21
5.2 Formal Setup and Notation . . . 21
5.3 Main Ideas . . . 21
5.4 Worked Example . . . 21
5.5 Systems and Implementation Consequences . . . 21
5.6 Common Misconceptions . . . 22
5.7 Exercises . . . 22
5.8 Further Reading . . . 22

6 Minimal Linguistics Background 23
6.1 Big Picture . . . 23
6.2 Formal Setup and Notation . . . 23
6.3 Main Ideas . . . 23
6.4 Worked Example . . . 23

6.5 Systems and Implementation Consequences . . . 23
6.6 Common Misconceptions . . . 24
6.7 Exercises . . . 24
6.8 Further Reading . . . 24

II Language Modeling Foundations 25

7 The Language Modeling Problem 27
7.1 Big Picture . . . 27
7.2 Formal Setup and Notation . . . 28
7.3 Autoregressive Decomposition . . . 28
7.4 Scoring versus Generation . . . 28
7.4.1 Scoring . . . 29
7.4.2 Generation . . . 29
7.5 Worked Example: Computing Sequence Probability . . . 29
7.6 Systems and Implementation Consequences . . . 30
7.7 Common Misconceptions . . . 30
7.8 Exercises . . . 31
7.9 Further Reading . . . 31

8 Tokenization 33
8.1 Big Picture . . . 33
8.2 Formal Setup and Notation . . . 34
8.3 Byte-Pair Encoding (BPE) . . . 34
8.3.1 The BPE Training Algorithm . . . 34
8.3.2 Encoding with BPE . . . 35
8.4 The Unigram Language Model Tokenizer . . . 35
8.5 Worked Example: BPE Training . . . 35
8.6 Systems and Implementation Consequences . . . 36
8.7 Common Misconceptions . . . 37
8.8 Exercises . . . 38
8.9 Further Reading . . . 38

9 From N-Grams to Neural Language Models 39
9.1 Big Picture . . . 39
9.2 Formal Setup and Notation . . . 40
9.3 The N-Gram Language Model . . . 40
9.3.1 The Curse of Dimensionality . . . 40
9.4 Worked Example: Trigram Calculation . . . 40

9.5 The Neural Probabilistic Language Model (NPLM) . . . 41
9.6 The Recurrent Era and Its Failure Modes . . . 41
9.7 Systems and Implementation Consequences . . . 42
9.8 Common Misconceptions . . . 43
9.9 Exercises . . . 43
9.10 Further Reading . . . 44

III Transformer Internals **45**

10 Attention Mechanisms **47**
10.1 Big Picture . . . 47
10.2 Formal Setup and Notation . . . 47
10.3 Main Ideas . . . 47
10.4 Worked Example . . . 47
10.5 Systems and Implementation Consequences . . . 47
10.6 Common Misconceptions . . . 48
10.7 Exercises . . . 48
10.8 Further Reading . . . 48

11 Transformer Blocks from Scratch **49**
11.1 Big Picture . . . 49
11.2 Formal Setup and Notation . . . 49
11.3 Main Ideas . . . 49
11.4 Worked Example . . . 49
11.5 Systems and Implementation Consequences . . . 49
11.6 Common Misconceptions . . . 50
11.7 Exercises . . . 50
11.8 Further Reading . . . 50

12 Norms, Residuals, and Positional Encodings **51**
12.1 Big Picture . . . 51
12.2 Formal Setup and Notation . . . 51
12.3 Main Ideas . . . 51
12.4 Worked Example . . . 51
12.5 Systems and Implementation Consequences . . . 52
12.6 Common Misconceptions . . . 52
12.7 Exercises . . . 52
12.8 Further Reading . . . 52

13 Hyperparameters, Initialization, and Optimization **53**
13.1 Big Picture . . . 53
13.2 Formal Setup and Notation . . . 53
13.3 Main Ideas . . . 53
13.4 Worked Example . . . 53
13.5 Systems and Implementation Consequences . . . 54
13.6 Common Misconceptions . . . 54
13.7 Exercises . . . 54
13.8 Further Reading . . . 54

14 Architectural Variants and Extensions **55**
14.1 Big Picture . . . 55
14.2 Formal Setup and Notation . . . 55
14.3 Main Ideas . . . 55
14.4 Worked Example . . . 55
14.5 Systems and Implementation Consequences . . . 56
14.6 Common Misconceptions . . . 56
14.7 Exercises . . . 56
14.8 Further Reading . . . 56

IV Systems and Implementation **57**

15 Resource Accounting **59**
15.1 Big Picture . . . 59
15.2 Formal Setup and Notation . . . 59
15.3 Main Ideas . . . 59
15.4 Worked Example . . . 59
15.5 Systems and Implementation Consequences . . . 59
15.6 Common Misconceptions . . . 60
15.7 Exercises . . . 60
15.8 Further Reading . . . 60

16 GPUs, TPUs, and Memory Hierarchy **61**
16.1 Big Picture . . . 61
16.2 Formal Setup and Notation . . . 61
16.3 Main Ideas . . . 61
16.4 Worked Example . . . 61
16.5 Systems and Implementation Consequences . . . 61
16.6 Common Misconceptions . . . 62

16.7 Exercises . . . 62
16.8 Further Reading . . . 62

17 Kernels, CUDA, Triton, and FlashAttention **63**
17.1 Big Picture . . . 63
17.2 Formal Setup and Notation . . . 63
17.3 Main Ideas . . . 63
17.4 Worked Example . . . 63
17.5 Systems and Implementation Consequences . . . 64
17.6 Common Misconceptions . . . 64
17.7 Exercises . . . 64
17.8 Further Reading . . . 64

18 Parallelism and Distributed Training **65**
18.1 Big Picture . . . 65
18.2 Formal Setup and Notation . . . 65
18.3 Main Ideas . . . 65
18.4 Worked Example . . . 65
18.5 Systems and Implementation Consequences . . . 65
18.6 Common Misconceptions . . . 66
18.7 Exercises . . . 66
18.8 Further Reading . . . 66

19 Inference and Serving **67**
19.1 Big Picture . . . 67
19.2 Formal Setup and Notation . . . 67
19.3 Main Ideas . . . 67
19.4 Worked Example . . . 67
19.5 Systems and Implementation Consequences . . . 67
19.6 Common Misconceptions . . . 68
19.7 Exercises . . . 68
19.8 Further Reading . . . 68

V Scaling, Evaluation, and Recipes **69**

20 Scaling Laws **71**
20.1 Big Picture . . . 71
20.2 Formal Setup and Notation . . . 71
20.3 Main Ideas . . . 71

20.4 Worked Example . . . 71
20.5 Systems and Implementation Consequences . . . 71
20.6 Common Misconceptions . . . 72
20.7 Exercises . . . 72
20.8 Further Reading . . . 72

21 Model Recipes and Open-Model Case Studies **73**
21.1 Big Picture . . . 73
21.2 Formal Setup and Notation . . . 73
21.3 Main Ideas . . . 73
21.4 Worked Example . . . 73
21.5 Systems and Implementation Consequences . . . 74
21.6 Common Misconceptions . . . 74
21.7 Exercises . . . 74
21.8 Further Reading . . . 74

22 Evaluation **75**
22.1 Big Picture . . . 75
22.2 Formal Setup and Notation . . . 75
22.3 Main Ideas . . . 75
22.4 Worked Example . . . 75
22.5 Systems and Implementation Consequences . . . 75
22.6 Common Misconceptions . . . 76
22.7 Exercises . . . 76
22.8 Further Reading . . . 76

VI Data **77**

23 Data Sources, Crawling, and Cleaning **79**
23.1 Big Picture . . . 79
23.2 Formal Setup and Notation . . . 79
23.3 Main Ideas . . . 79
23.4 Worked Example . . . 79
23.5 Systems and Implementation Consequences . . . 79
23.6 Common Misconceptions . . . 80
23.7 Exercises . . . 80
23.8 Further Reading . . . 80

24 Filtering, Deduplication, and Data Mixing **81**
24.1 Big Picture . . . 81
24.2 Formal Setup and Notation . . . 81
24.3 Main Ideas . . . 81
24.4 Worked Example . . . 81
24.5 Systems and Implementation Consequences . . . 82
24.6 Common Misconceptions . . . 82
24.7 Exercises . . . 82
24.8 Further Reading . . . 82

VII Alignment and Reasoning **83**

25 Instruction Tuning and Preference Tuning **85**
25.1 Big Picture . . . 85
25.2 Formal Setup and Notation . . . 85
25.3 Main Ideas . . . 85
25.4 Worked Example . . . 85
25.5 Systems and Implementation Consequences . . . 86
25.6 Common Misconceptions . . . 86
25.7 Exercises . . . 86
25.8 Further Reading . . . 86

26 RLHF, DPO, GRPO, and Reasoning RL **87**
26.1 Big Picture . . . 87
26.2 Formal Setup and Notation . . . 87
26.3 Main Ideas . . . 87
26.4 Worked Example . . . 87
26.5 Systems and Implementation Consequences . . . 88
26.6 Common Misconceptions . . . 88
26.7 Exercises . . . 88
26.8 Further Reading . . . 88

27 Safety, Reward Hacking, and Deployment **89**
27.1 Big Picture . . . 89
27.2 Formal Setup and Notation . . . 89
27.3 Main Ideas . . . 89
27.4 Worked Example . . . 89
27.5 Systems and Implementation Consequences . . . 90
27.6 Common Misconceptions . . . 90

27.7 Exercises 90
27.8 Further Reading 90

A Proof Techniques and Derivations **93**

Appendices **93**

B CUDA and Triton Cookbook **95**

C Linguistics Quick Reference **97**

D Neuroscience Analogy Index **99**

E Reproducibility and Debugging **101**

References **103**

List of Figures

List of Tables

Part I

Orientation and Prerequisites

Chapter 1

How to Use This Book

This book accompanies Stanford's CS336: Language Modeling from Scratch. It follows the course's arc—from tokenization to alignment—but provides deeper mathematical treatment, more worked examples, and self-contained prerequisite coverage.

1.1 The "From Scratch" Philosophy

When we say "from scratch," we mean it. By the end of this book, you will have seen how to:

- Define the language modeling problem using information theory
- Build a tokenizer from raw text
- Construct a Transformer architecture layer by layer
- Train it efficiently on GPUs with custom kernels
- Scale it using empirical scaling laws
- Curate and filter training data
- Align the model to follow instructions and reason

No black boxes. No "just import this library." Every component is derived, explained, and connected to the underlying mathematics.

1.2 How to Read This Book

1.2.1 Sequential Reading

If you are new to language models, read Part 0 first to solidify prerequisites, then proceed through Parts I–VI in order. This is the recommended path.

1.2.2 "I Know ML" Path

If you are comfortable with probability, linear algebra, and PyTorch, skip Part 0 and start with Chapter 6 (The Language Modeling Problem). Refer back to Part 0 when needed.

1.2.3 Systems Focus

For a systems-oriented reading, focus on Chapters 14–18 after reviewing the Transformer basics in Chapters 9–11.

1.2.4 Assignment Companion

Each CS336 assignment maps to specific chapters:

Assignment	Topic	Chapters
A1	Basics	6–12
A2	Systems	14–17
A3	Scaling	19–21
A4	Data	22–23
A5	Alignment	24–26

1.3 Notation Conventions

We use the following conventions throughout the book:

- **Scalars** are lowercase italic: x, y, α
- **Vectors** are lowercase bold: $\mathbf{x}, \mathbf{y}$
- **Matrices** are uppercase bold: $\mathbf{W}, \mathbf{Q}, \mathbf{K}, \mathbf{V}$
- **Sets** are calligraphic: $\mathcal{V}$ (vocabulary), $\mathcal{D}$ (dataset)
- log always means natural logarithm unless otherwise noted
- Complete notation reference: see the Notation Table on page 5

1.4 Neuroscience Analogies

Throughout this book, you will encounter purple-shaded **Neuroscience Analogy** boxes. These connect machine learning concepts to brain science to build intuition. Every analogy box has three parts:

1. **Where the analogy helps**: the genuine structural or functional similarity

2. **Where the analogy breaks**: the important differences
3. **What intuition to keep**: the one-sentence takeaway

These analogies are thinking tools, not scientific claims about how LLMs work.

1.5 Notation Table

Symbol	**Meaning**
T	Sequence length (context window)
V or $\|\mathcal{V}\|$	Vocabulary size
d or d_{model}	Model / embedding dimension
d_k, d_v	Key and value dimensions per attention head
d_{ff}	Feed-forward hidden dimension
n_h	Number of attention heads
n_{kv}	Number of key-value heads (for GQA)
L	Number of Transformer layers
N	Number of model parameters
D	Dataset size (in tokens)
B	Batch size
η	Learning rate
$\boldsymbol{\theta}$	Parameter vector
$p(x)$	True / empirical distribution
$p_{\boldsymbol{\theta}}(x)$ or $q(x)$	Model distribution
$H(p)$	Entropy
$H(p, q)$	Cross-entropy
$D_{\text{KL}}(p \,\|\, q)$	KL divergence
PPL	Perplexity
$\text{softmax}(\cdot)$	Softmax function (row-wise for matrices)
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	Query, Key, Value matrices
C	Compute budget (FLOPs)
MFU	Model FLOPs Utilization

Chapter 2

Probability, Entropy, and Information Theory

Key Insight

After reading this chapter, you will be able to:

1. Define and compute entropy, cross-entropy, and KL divergence for discrete distributions
2. Derive the connection between maximum likelihood estimation and cross-entropy minimization
3. Explain perplexity and use it to evaluate language models
4. State precisely what a language model is as a probability distribution
5. Use these concepts to motivate why we train language models the way we do

2.1 Big Picture

Every language model is, at its core, a probability distribution over sequences of tokens. When we "train" a model, we are adjusting its parameters so that the probability distribution it represents comes closer to the true distribution of natural language. To make this precise, we need three foundational concepts from information theory:

1. **Entropy** tells us how much uncertainty a distribution has—how "surprised" we should be on average.
2. **Cross-entropy** measures how well one distribution approximates another, and gives us our training loss.

3. **KL divergence** quantifies the "gap" between two distributions, connecting training loss to the information-theoretic ideal.

These are not abstract mathematical curiosities. Cross-entropy is literally the loss function we minimize when training GPT, LLaMA, or any autoregressive language model. Understanding it deeply will make every subsequent chapter more intuitive.

This chapter connects directly to Shannon's 1950 paper on predicting English text [1], which is the intellectual ancestor of all language modeling work.

2.2 Probability Distributions over Tokens

Definition 2.1 (Discrete probability distribution)**.** Let $\mathcal{V} = \{v_1, v_2, \ldots, v_{|\mathcal{V}|}\}$ be a finite set (our **vocabulary**). A **probability distribution** over $\mathcal{V}$ is a function $p : \mathcal{V} \to [0, 1]$ such that

$$\sum_{v \in \mathcal{V}} p(v) = 1. \tag{2.1}$$

We write $p(v)$ for the probability assigned to token v.

In language modeling, $\mathcal{V}$ is our vocabulary of tokens (words, subwords, or characters—see Chapter 7). A language model assigns a probability to each possible next token given a context of previous tokens.

Example 2.1. Suppose our vocabulary is $\mathcal{V} = \{\texttt{the}, \texttt{cat}, \texttt{sat}, \texttt{on}, \texttt{mat}\}$ and after seeing the context "the cat sat on the," our model assigns:

Token v	$p(v \mid$ "the cat sat on the"$)$
`mat`	0.62
`cat`	0.15
`the`	0.08
`sat`	0.10
`on`	0.05

This is a valid probability distribution: all values are in $[0, 1]$ and they sum to 1.00.

2.3 Entropy: Measuring Uncertainty

2.3.1 Definition and Intuition

How "uncertain" or "surprising" is a probability distribution? Shannon's entropy answers this precisely.

Definition 2.2 (Entropy)**.** The **entropy** of a discrete probability distribution p over a finite set $\mathcal{V}$ is

$$H(p) = -\sum_{v \in \mathcal{V}} p(v) \log p(v), \tag{2.2}$$

where log denotes the natural logarithm and we adopt the convention $0 \log 0 = 0$.

Symbol table. $H(p)$ is the entropy (a non-negative real number); $p(v)$ is the probability of token v; the sum is over all tokens in the vocabulary $\mathcal{V}$.

Interpretation. Entropy measures the average "surprise" or "information content" when we draw a token from p. The quantity $-\log p(v)$ is called the **surprisal** of token v: rare events (low $p(v)$) have high surprisal, and common events have low surprisal. Entropy is the expected surprisal.

2.3.2 Key Properties

Proposition 2.1 (Bounds on entropy)**.** *For any distribution p over $\mathcal{V}$ with $|\mathcal{V}| = V$:*

$$0 \leq H(p) \leq \log V. \tag{2.3}$$

- $H(p) = 0$ *if and only if p is a point mass (all probability on one token—zero uncertainty).*
- $H(p) = \log V$ *if and only if p is the uniform distribution (maximum uncertainty).*

Proof. The lower bound follows because $-p(v) \log p(v) \geq 0$ for all v. For the upper bound, we use the fact that log is concave. By Jensen's inequality:

$$H(p) = \mathbb{E}_{v \sim p}[-\log p(v)] \leq -\log\left(\mathbb{E}_{v \sim p}[p(v)]\right) = -\log\left(\sum_{v} p(v)^2\right) \leq \log V, \tag{2.4}$$

with equality when $p(v) = 1/V$ for all v. Alternatively, one can verify this using Lagrange multipliers or the log-sum inequality. □

2.3.3 Worked Example: Computing Entropy

Consider a bigram language model over $\mathcal{V} = \{a, b, c\}$ that predicts the next character with distribution $p(a) = 0.7$, $p(b) = 0.2$, $p(c) = 0.1$:

$$
\begin{aligned}
H(p) &= -[0.7 \log 0.7 + 0.2 \log 0.2 + 0.1 \log 0.1] && (2.5)\\
&= -[0.7 \times (-0.357) + 0.2 \times (-1.609) + 0.1 \times (-2.303)] && (2.6)\\
&= -[-0.250 - 0.322 - 0.230] && (2.7)\\
&= 0.802 \text{ nats.} && (2.8)
\end{aligned}
$$

For comparison, the uniform distribution over 3 tokens has entropy $\log 3 \approx 1.099$ nats. Our distribution has lower entropy because it concentrates most probability on token a—it is more "certain" about what comes next.

Sanity check. The entropy 0.802 is between 0 (point mass) and 1.099 (uniform), as expected.

2.4 Cross-Entropy: Measuring Model Quality

2.4.1 Definition

In language modeling, we have two distributions:

- p: the true distribution of language (unknown, but we have samples from it)
- q (or $p_{\boldsymbol{\theta}}$): our model's predicted distribution

We want to measure how well q approximates p.

Definition 2.3 (Cross-entropy)**.** The **cross-entropy** of distribution q relative to distribution p is

$$H(p, q) = -\sum_{v \in \mathcal{V}} p(v) \log q(v) = \mathbb{E}_{v \sim p}[-\log q(v)]. \tag{2.9}$$

Symbol table. $p(v)$ is the probability of token v under the true distribution; $q(v)$ is the probability under our model; $-\log q(v)$ is the surprisal that our model assigns to token v.

Interpretation. Cross-entropy is the average surprisal when events are drawn from p but we use model q to predict them. If $q = p$, then $H(p, q) = H(p)$—the best we can do. If q diverges from p, cross-entropy increases.

2.4.2 Cross-Entropy as a Training Loss

In practice, we do not know p exactly. Instead, we have a training corpus $\mathcal{D} = (x_1, x_2, \ldots, x_n)$ of n tokens drawn (approximately) from p. The empirical cross-entropy is:

$$\hat{H}(p, q) = -\frac{1}{n} \sum_{i=1}^{n} \log q(x_i \mid x_1, \ldots, x_{i-1}). \tag{2.10}$$

This is exactly the average negative log-likelihood per token—the standard training loss for autoregressive language models such as GPT, LLaMA, and DeepSeek.

Key Insight

When you see a language model training with "cross-entropy loss," it is minimizing exactly Equation (2.10): the average surprisal of the training data under the model. Lower is better.

2.5 KL Divergence: The Gap Between Distributions

Definition 2.4 (KL divergence)**.** The **Kullback–Leibler divergence** from distribution p to distribution q is

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_{v \in \mathcal{V}} p(v) \log \frac{p(v)}{q(v)} = H(p, q) - H(p). \tag{2.11}$$

Interpretation. KL divergence is the "extra cost" (in nats) of using model q instead of the true distribution p. It decomposes cross-entropy into two parts:

$$\underbrace{H(p, q)}_{\text{cross-entropy (what we minimize)}} = \underbrace{H(p)}_{\text{entropy of data (constant)}} + \underbrace{D_{\mathrm{KL}}(p \,\|\, q)}_{\text{gap (what shrinks during training)}}. \tag{2.12}$$

Since $H(p)$ does not depend on the model parameters $\boldsymbol{\theta}$, minimizing cross-entropy $H(p, p_{\boldsymbol{\theta}})$ is equivalent to minimizing $D_{\mathrm{KL}}(p \,\|\, p_{\boldsymbol{\theta}})$.

Proposition 2.2 (Gibbs' inequality)**.** *For any distributions p and q over the same finite set:*

$$D_{\mathrm{KL}}(p \,\|\, q) \geq 0, \tag{2.13}$$

with equality if and only if $p = q$.

Proof. By Jensen's inequality applied to the convex function $-\log$:

$$D_{\mathrm{KL}}(p \,\|\, q) = -\sum_{v} p(v) \log \frac{q(v)}{p(v)} \geq -\log \left(\sum_{v} p(v) \cdot \frac{q(v)}{p(v)} \right) = -\log \left(\sum_{v} q(v) \right) = -\log 1 = 0. \tag{2.14}$$

Equality holds iff $q(v)/p(v)$ is constant for all v with $p(v) > 0$, i.e., $p = q$. □

Common Pitfall

KL divergence is not symmetric: $D_{\mathrm{KL}}(p \,\|\, q) \neq D_{\mathrm{KL}}(q \,\|\, p)$ in general. It is not a metric (distance) in the mathematical sense. In language modeling, we always minimize $D_{\mathrm{KL}}(p \,\|\, p_{\boldsymbol{\theta}})$ (forward KL), not $D_{\mathrm{KL}}(p_{\boldsymbol{\theta}} \,\|\, p)$ (reverse KL). The choice matters: forward KL encourages the model to cover all modes of p (it is "mean-seeking"), while reverse KL encourages the model to concentrate on the highest-probability mode (it is "mode-seeking").

2.6 Maximum Likelihood and Cross-Entropy

We now show that the standard training objective for language models—maximum likelihood estimation—is equivalent to cross-entropy minimization.

Proposition 2.3 (MLE = Cross-entropy minimization)**.** *Given a training corpus $\mathcal{D} = (x_1, \ldots, x_n)$, the maximum likelihood estimate*

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{n} \log p_{\boldsymbol{\theta}}(x_i \mid x_{<i}) \tag{2.15}$$

is equivalent to

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \hat{H}(p_{data}, p_{\boldsymbol{\theta}}). \tag{2.16}$$

Proof. Maximizing $\sum_i \log p_{\boldsymbol{\theta}}(x_i \mid x_{<i})$ is the same as minimizing $-\frac{1}{n}\sum_i \log p_{\boldsymbol{\theta}}(x_i \mid x_{<i})$, which is exactly the empirical cross-entropy $\hat{H}(p_{\text{data}}, p_{\boldsymbol{\theta}})$ from Equation (2.10). The scaling factor $1/n$ does not affect the argmin. □

Key Insight

This is why "next-token prediction" works as a training objective. By predicting the next token as accurately as possible (maximizing likelihood), we are simultaneously minimizing the information-theoretic gap between our model and natural language.

2.7 Perplexity: The Standard Evaluation Metric

Definition 2.5 (Perplexity)**.** The **perplexity** of a model q on a sequence $(x_1, \ldots, x_T)$ is

$$\mathrm{PPL}(q) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log q(x_t \mid x_{<t})\right) = \exp\left(\hat{H}(p, q)\right). \tag{2.17}$$

Interpretation. Perplexity is the exponential of cross-entropy. It can be interpreted as the **effective vocabulary size**: a perplexity of 100 means the model is "as confused" as if it were choosing uniformly among 100 equally likely tokens at each step.

Key relationships.

- A perfect model ($q = p$) achieves PPL $= \exp(H(p))$—the irreducible perplexity determined by the entropy of language itself.
- Lower perplexity = better model.
- Perplexity is the standard metric reported in language modeling papers (e.g., GPT-2, LLaMA).
- If using $\log_2$ instead of ln, we get PPL $= 2^{H_2(p,q)}$ where H_2 is cross-entropy in bits.

2.7.1 Worked Example: Perplexity Calculation

Suppose our model assigns the following probabilities to a 5-token test sequence:

Position t	Token x_t	$q(x_t \mid x_{<t})$
1	`the`	0.20
2	`cat`	0.05
3	`sat`	0.30
4	`on`	0.50
5	`mat`	0.60

Then:

$$\text{avg. NLL} = -\frac{1}{5}\left[\log 0.20 + \log 0.05 + \log 0.30 + \log 0.50 + \log 0.60\right] \tag{2.18}$$

$$= -\frac{1}{5}\left[-1.609 + (-2.996) + (-1.204) + (-0.693) + (-0.511)\right] \tag{2.19}$$

$$= -\frac{1}{5}(-7.013) = 1.403 \tag{2.20}$$

$$\text{PPL} = \exp(1.403) \approx 4.07. \tag{2.21}$$

A perplexity of 4.07 means the model is, on average, as uncertain as if it were choosing uniformly among about 4 tokens—not bad for a vocabulary that might contain tens of thousands of tokens.

Sanity check. The geometric mean of the per-token probabilities is $(0.20 \times 0.05 \times 0.30 \times 0.50 \times 0.60)^{1/5} \approx 0.245$, and $1/0.245 \approx 4.07$. This confirms: perplexity is the reciprocal of the geometric mean probability.

2.8 Shannon's Game and the Entropy Rate of English

In 1950, Claude Shannon asked: how much information does each letter in English carry [1]? He devised a guessing game where a human predicts the next character in a text, and used the results to estimate the entropy rate of English at approximately 1.0–1.5 bits per character.

This experiment is the direct ancestor of modern language model evaluation:

- Shannon's human predictor $\longrightarrow$ our neural language model $p_{\boldsymbol{\theta}}$
- Shannon's guessing accuracy $\longrightarrow$ our model's perplexity
- Shannon's entropy bound $\longrightarrow$ the irreducible loss floor

Modern language models like GPT-4 approach or surpass human-level prediction accuracy on many text domains, suggesting they have learned much of the statistical structure of language. But as we will see in later chapters, statistical prediction is not the same as understanding.

Neuroscience Analogy: Predictive Coding in the Brain

Where the analogy helps. The brain constantly generates predictions about upcoming sensory input. When the actual input matches the prediction, neural activity is low (low surprise). When the input is unexpected, prediction error signals propagate up the cortical hierarchy. This is strikingly parallel to how a language model computes surprisal $-\log p_{\boldsymbol{\theta}}(x_t \mid x_{<t})$: expected tokens incur low loss, surprising tokens incur high loss.

Where the analogy breaks. The brain's predictive coding operates across multiple modalities (vision, touch, proprioception) simultaneously, uses lateral and feedback connections, and updates predictions in real time with no distinct "training phase." A Transformer language model processes text only, uses feedforward computation (with residual connections, not true recurrence), and has a sharp divide between training and inference.

What intuition to keep. Think of next-token prediction loss as a measure of the model's "surprise"—exactly like prediction error signals in the brain. Training reduces this surprise by learning the statistical patterns of language. But the model's "prediction" is purely statistical, not a theory of the world.

2.9 Common Misconceptions

1. **"Lower cross-entropy always means a better model."** Only when comparing models on the same data with the same tokenization. Different tokenizations change the token count T, making cross-entropy values incomparable. Always compare with matched tokenization or use bits-per-byte.

2. **"KL divergence is a distance metric."** It is not: $D_{\mathrm{KL}}(p \,\|\, q) \neq D_{\mathrm{KL}}(q \,\|\, p)$ in general, and it does not satisfy the triangle inequality. Treat it as a directed measure of information loss.

3. **"Perplexity of 1 means a perfect model."** A perplexity of 1 would mean the model predicts every token with probability 1—this is impossible for natural language, which has genuine uncertainty. A "perfect" model would achieve $\mathrm{PPL} = \exp(H(p))$, which is greater than 1.

4. **"Cross-entropy loss is just a convention; we could use MSE."** Cross-entropy is the uniquely correct loss for maximum likelihood training of categorical distributions. Using MSE would correspond to a different (and generally worse) probabilistic assumption about the output.

5. **"Shannon showed that English has about 1 bit of entropy per character, so LLMs are almost done."** Shannon estimated the *entropy rate per character*. Modern subword models operate over much larger token units. The entropy rate of language depends on the unit of measurement, and there is still substantial room for improvement, especially for rare and specialized language.

2.10 Exercises

Warmup

1. Compute the entropy of the distribution $p(a) = 0.5, p(b) = 0.25, p(c) = 0.25$ in both nats and bits.

2. A model assigns probability 0.1 to a token that actually appeared. What is the surprisal in nats? In bits?

3. If a model has perplexity 50 on a test set, what is its average cross-entropy loss per token (in nats)?

Derivation

1. Prove that among all distributions over $\{1, 2, \ldots, V\}$, the uniform distribution uniquely maximizes entropy. (Hint: use Lagrange multipliers or the log-sum inequality.)

2. Show that $D_{\mathrm{KL}}(p \,\|\, q) \geq 0$ using the log-sum inequality instead of Jensen's inequality.

3. Derive the relationship between perplexity and the geometric mean of per-token probabilities: $\mathrm{PPL} = \left(\prod_{t=1}^{T} q(x_t \mid x_{<t})\right)^{-1/T}$.

Implementation

1. [**Prepares for CS336 A1**] Write a Python function `compute_perplexity(log_probs: list[float]) -> float` that takes a list of log-probabilities $\log q(x_t \mid x_{<t})$ and returns the perplexity of the sequence. Test it on the worked example from Section 2.7.

2. Write a function that takes two discrete probability distributions (as NumPy arrays) and computes their cross-entropy and KL divergence. Verify that $D_{\mathrm{KL}}(p \,\|\, q) = H(p, q) - H(p)$.

Open-Ended

1. Shannon's 1950 experiment used human predictors. How would you design a modern version of this experiment using a language model? What would change if you used GPT-4 instead of a human? What aspects of language might a human still predict better than a model?

2.11 Further Reading

- "Prediction and Entropy of Printed English" [1]: The foundational paper on predicting English text. Introduces the entropy rate experiments that motivate all language modeling work. Remarkably readable for a 1950 paper.

- Cover & Thomas, *Elements of Information Theory*, Chapter 2: A rigorous treatment of entropy, relative entropy (KL divergence), and mutual information. The standard reference for information theory.

- MacKay, *Information Theory, Inference, and Learning Algorithms*, Chapters 2–4: A more intuitive and example-rich treatment of the same material, freely available online.

- "A Neural Probabilistic Language Model" [2]: The paper that introduced neural language models. Uses cross-entropy training as described in this chapter. Reading it shows how the information-theoretic framework connects to neural architectures.

Chapter 3

Linear Algebra and Matrix Calculus

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

3.1 Big Picture

TODO: Situate this chapter in the overall narrative.

3.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

3.3 Main Ideas

TODO: Core technical content.

3.4 Worked Example

TODO: Complete worked example.

3.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

3.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

3.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

3.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 4

PyTorch and Tensor Computation

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

4.1 Big Picture

TODO: Situate this chapter in the overall narrative.

4.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

4.3 Main Ideas

TODO: Core technical content.

4.4 Worked Example

TODO: Complete worked example.

4.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

4.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

4.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

4.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 5

CUDA and GPU Execution

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

5.1 Big Picture

TODO: Situate this chapter in the overall narrative.

5.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

5.3 Main Ideas

TODO: Core technical content.

5.4 Worked Example

TODO: Complete worked example.

5.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

5.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

5.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

5.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 6

Minimal Linguistics Background

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

6.1 Big Picture

TODO: Situate this chapter in the overall narrative.

6.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

6.3 Main Ideas

TODO: Core technical content.

6.4 Worked Example

TODO: Complete worked example.

6.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

6.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

6.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

6.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part II

Language Modeling Foundations

Chapter 7

The Language Modeling Problem

Key Insight

After reading this chapter, you will be able to:

1. State the formal definition of autoregressive language modeling.
2. Explain how the chain rule of probability decomposes sequence modeling into next-token prediction.
3. Differentiate between generation (sampling) and scoring (likelihood calculation).
4. Understand the fundamental bottleneck of language modeling: context dependency versus computational feasibility.

7.1 Big Picture

Everything in natural language processing experienced a paradigm shift when researchers realized a profound, counterintuitive truth: predicting the next word in a sequence is not just a parlor trick. It is a conceptually simple objective that, when scaled up, forces a neural network to learn syntax, semantics, logic, and common sense.

To accurately predict the next word in "The cat sat on the ____", a model only needs to know English grammar and common idioms. But to predict the next word in "If we heat the metal to 2000°C, its tensile strength will ____", the model must understand physics and materials science. By training a single architecture to predict the next token across all of human text, we obtain a general-purpose reasoning engine.

This chapter forms the conceptual bridge from the information theory in Chapter 1 to the neural implementations in Chapters 8 and beyond. We establish the fundamental mathematical problem that all generative language models—from n-grams to GPT-4—solve.

7.2 Formal Setup and Notation

We treat language as a sequence of discrete units called **tokens**. (We will discuss exactly how text is split into tokens in Chapter 7; for now, consider them words or characters).

Let our vocabulary be a finite set $\mathcal{V}$. A sequence of length T is denoted $\mathbf{x} = (x_1, x_2, \ldots, x_T)$, where each $x_t \in \mathcal{V}$. The language modeling problem is to estimate the joint probability distribution of the entire sequence:

$$P(X_1 = x_1, X_2 = x_2, \ldots, X_T = x_T) = P(\mathbf{x}). \tag{7.1}$$

Here, X_t is the random variable representing the token at position t, and x_t is the specific realization of that random variable.

7.3 Autoregressive Decomposition

Directly estimating $P(\mathbf{x})$ is intractable. If our vocabulary has $|\mathcal{V}| = 50,000$ and we consider sequences of length $T = 100$, there are $50,000^{100}$ possible sequences. We cannot simply count frequencies in a training corpus; almost all sequences will occur exactly zero times.

To make the problem tractable, we apply the **chain rule of probability**:

$$P(x_1, x_2, \ldots, x_T) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2) \cdots P(x_T \mid x_1, \ldots, x_{T-1}). \tag{7.2}$$

Using product notation:

$$P(\mathbf{x}) = \prod_{t=1}^{T} P(x_t \mid x_{<t}), \tag{7.3}$$

where $x_{<t} = (x_1, x_2, \ldots, x_{t-1})$ represents the **context** or **history** up to position t.

Symbol table. $P(\mathbf{x})$ is the joint probability of the sequence. $P(x_t \mid x_{<t})$ is the conditional probability of the next token given all previous tokens.

Interpretation. The chain rule tells us that modeling an entire sequence is mathematically equivalent to modeling a single next token, provided we condition on all previous tokens. This transforms language modeling from a massive joint distribution problem into a sequence of classification problems. A model that predicts one step at a time and feeds its own predictions back as input is called **autoregressive**.

7.4 Scoring versus Generation

An autoregressive language model has two primary uses: scoring existing text and generating new text.

7.4.1 Scoring

We can use a language model to compute the likelihood of a given sequence. Recall from Chapter 1 that we typically work with log-probabilities to avoid arithmetic underflow:

$$\log P(\mathbf{x}) = \sum_{t=1}^{T} \log P(x_t \mid x_{<t}). \tag{7.4}$$

This is the heart of model evaluation (perplexity) and training (cross-entropy loss minimization).

7.4.2 Generation

To generate text, we sample from the model one token at a time:

1. Start with a prompt context $x_{<1}$.
2. Sample $x_1 \sim P(\cdot \mid x_{<1})$.
3. Append x_1 to the context: $x_{<2} = (x_{<1}, x_1)$.
4. Sample $x_2 \sim P(\cdot \mid x_{<2})$.
5. Repeat until a special stop token is generated.

7.5 Worked Example: Computing Sequence Probability

Suppose our vocabulary is $\mathcal{V} = \{\text{the}, \text{cat}, \text{sat}\}$. We want to score the sequence $\mathbf{x} = (\text{the}, \text{cat}, \text{sat})$. Our model provides the following conditional probabilities:

$$\begin{aligned} P(x_1 = \text{the}) &= 0.2 \\ P(x_2 = \text{cat} \mid x_1 = \text{the}) &= 0.4 \\ P(x_3 = \text{sat} \mid x_1 = \text{the}, x_2 = \text{cat}) &= 0.8 \end{aligned}$$

The joint probability of the sequence is:

$$\begin{aligned} P(\mathbf{x}) &= P(\text{the}) \times P(\text{cat} \mid \text{the}) \times P(\text{sat} \mid \text{the}, \text{cat}) \\ &= 0.2 \times 0.4 \times 0.8 = 0.064 \end{aligned}$$

In minus log-likelihood (base e):

$$\begin{aligned} -\log(0.064) &= -(\log(0.2) + \log(0.4) + \log(0.8)) \\ &= -(-1.609 - 0.916 - 0.223) = 2.748 \text{ nats} \end{aligned}$$

7.6 Systems and Implementation Consequences

The autoregressive decomposition Equation (7.3) dictates the computational architecture of all modern generative AI.

During training (scoring): Because we are scoring a known sequence $x_1, \ldots, x_T$, the context $x_{<t}$ is fixed for all t. This means we can compute the probabilities for all T positions in parallel. This parallelism is the defining advantage of the Transformer architecture over older recurrent networks.

During inference (generation): Because we must wait to sample x_t before we can append it to the context and predict x_{t+1}, generation is inherently sequential. This makes decoding highly latency-bound and memory bandwidth-bound. We cannot parallelize across time steps during generation. This dichotomy—parallel training, serial inference—drives the engineering constraints discussed in Chapters 14–18.

Neuroscience Analogy: Predictive Coding and Language Generation

Where the analogy helps. The human brain has been theorized to function as a "prediction machine," constantly anticipating upcoming sensory input. Similarly, as you read this sentence, your brain is likely anticipating the next word before your eyes reach it. An autoregressive language model formalizes this exact hypothesis into a mathematical framework.

Where the analogy breaks. Human language generation is hierarchical and goal-directed. A speaker starts with an abstract communicative intent, forms a syntactic chunk, and then articulates words. An autoregressive LLM, by contrast, operates purely left-to-right at the token level, with no explicit internal mechanism separating high-level intent from low-level token choice.

What intuition to keep. Think of the language modeling objective as pure prediction. The model learns everything it knows about grammar and reasoning simply as a byproduct of trying to reduce surprise about the future.

7.7 Common Misconceptions

1. **"Language models understand text, then generate it."** Language models are function approximators mapping contexts to probability distributions. Any semantic "understanding" is entirely implicit in the structure of the conditional distribution $P(x_t \mid x_{<t})$.

2. **"The model predicts the most likely entire sequence."** Usually, models perform greedy decoding (picking the highest probability token at each step) or sampling. Finding the global maximum probability sequence over the entire sequence length is intractable.

3. **"Generative modeling algorithms are specific to text."** The autoregressive formulation applies to any sequence. The exact same objective is used for image generation (treating images as sequences of pixels) and audio generation (as sequences of audio waveforms).

7.8 Exercises

Warmup

1. Suppose a model assigns a probability of 0.1 to the true next token at every position in a sequence of length 5. What is the overall joint probability of the sequence? What is the log-probability?

Derivation

1. Show that if x_t is completely independent of the context $x_{<t}$, the autoregressive model reduces to a unigram model where $P(\mathbf{x}) = \prod_{t=1}^{T} P(x_t)$. What is the context size required to capture all dependencies in natural language?

Implementation

1. [**Prepares for CS336 A1**] Implement a basic loop in Python that demonstrates autoregressive generation given a dummy prediction function `def predict_next_token(context): return token`.

Open-Ended

1. Discuss why bidirectional models (like BERT, which predict missing words given both left and right context) are popular for classification tasks but struggle mathematically with the task of text generation. How does the chain rule map to bidirectionality?

7.9 Further Reading

- "A Neural Probabilistic Language Model" [2]: A Neural Probabilistic Language Model. The seminal paper framing language modeling as a neural task based on the Markov assumption.
- OpenAI [3]: Language Models are Unsupervised Multitask Learners. This OpenAI paper demonstrated that autoregressive next-token prediction, scaled massively, naturally gives rise to zero-shot task performance.

Chapter 8

Tokenization

Key Insight

After reading this chapter, you will be able to:

1. Explain why language models operate on discrete tokens rather than raw text.
2. Describe the trade-offs between word, character, and subword tokenization.
3. Implement the Byte-Pair Encoding (BPE) algorithm from scratch.
4. Understand the Unigram tokenization algorithm and its probabilistic formulation.
5. Discuss the motivation and architecture of byte-level and token-free models.

8.1 Big Picture

Before a neural network can process text, the text must be converted into numbers. This process is called **tokenization**. It sits at the very edge of the model, transforming a continuous stream of characters or bytes into a sequence of discrete integers.

Historically, NLP systems used words as the fundamental unit of text. However, natural language is fraught with complications: morphology ("run", "running", "ran"), agglutination (languages like German or Turkish that glue words together), and typos. If we tokenize by words, our vocabulary grows uncontrollably, leading to the "out-of-vocabulary" (OOV) problem. If we tokenize by characters, the vocabulary is small (e.g., 256 for ASCII), but sequences become extremely long, which computationally burdens the Transformer architecture.

The modern solution is **subword tokenization**. Algorithms like Byte-Pair Encoding (BPE) strike a data-driven balance: common words remain single tokens, while rare words are broken down into smaller, frequent chunks (or individual characters). Nearly every major language model today (GPT-4, LLaMA, DeepSeek) uses a variant of subword tokenization.

8.2 Formal Setup and Notation

Let $\mathcal{C}$ be our base character set (e.g., all 256 possible bytes, or all Unicode characters). A **corpus** is a string sequence over $\mathcal{C}$.

A **tokenizer** consists of two functions:

1. An **encoder**: $E : \mathcal{C}^* \to \mathcal{V}^*$, mapping a string of characters to a sequence of token IDs from a fixed integer vocabulary $\mathcal{V} = \{1, 2, \ldots, V\}$.

2. A **decoder**: $D : \mathcal{V}^* \to \mathcal{C}^*$, mapping a sequence of token IDs back to a string.

A proper tokenizer must be lossless: for any valid text s, $D(E(s)) = s$.

Vocabulary size. The size of the vocabulary V is a crucial hyperparameter. Typical values range from $32,000$ (LLaMA-2) to $128,000$ (GPT-4). A larger vocabulary means shorter sequences (good for compute) but requires a larger embedding matrix (bad for memory).

8.3 Byte-Pair Encoding (BPE)

BPE was originally a data compression algorithm introduced by "Sennrich" [4] to the NLP world. It is a greedy, bottom-up algorithm that builds a vocabulary by iteratively merging the most frequent pairs of units.

8.3.1 The BPE Training Algorithm

To "train" a BPE tokenizer on a corpus, we perform the following steps:

1. Initialize the vocabulary with the base vocabulary $\mathcal{C}$ (usually all 256 ASCII/UTF-8 bytes).

2. Represent the training corpus as a sequence of these base tokens.

3. Count the frequency of all adjacent token pairs.

4. Find the most frequent pair (t_1, t_2).

5. Create a new token $t_{\text{new}} = t_1 \circ t_2$ by concatenating them.

6. Add t_{new} to the vocabulary.

7. Replace all non-overlapping occurrences of the pair (t_1, t_2) in the corpus with t_{new}.

8. Repeat steps 3–7 until the vocabulary reaches the desired size V.

8.3.2 Encoding with BPE

Once trained, the sequence of merges defines the encoding process. Given a new string, we first split it into base tokens (bytes). We then apply the merge operations in the exact same order they were learned during training.

Common Pitfall

Order matters! If "e" + "r" → "er" was learned before "t" + "h" → "th", you must apply the "er" merge to your new text before looking for "th".

8.4 The Unigram Language Model Tokenizer

While BPE is bottom-up and greedy, the **Unigram** tokenizer operates top-down and probabilistically.

1. Initialize a massive vocabulary of potential subwords (e.g., all substrings up to length L in the corpus).
2. Train a unigram language model over these subwords. Let $p(x_i)$ be the probability of subword x_i. The probability of a tokenization $\mathbf{x} = (x_1, \ldots, x_T)$ is assumed to be $\prod p(x_i)$.
3. Find the optimal tokenization for the corpus (using the Viterbi algorithm) to maximize likelihood under the current probabilities.
4. Compute the "loss" for each subword: how much does the overall log-likelihood drop if we remove this subword from the vocabulary?
5. Discard the bottom 20% of subwords (those with the smallest loss).
6. Repeat until the vocabulary shrinks to the desired size V.

Unigram tokenization has the advantage of supporting **subword regularization**: because it is probabilistically grounded, we can sample different valid tokenizations for the same word during training, which makes the model more robust to misspellings.

8.5 Worked Example: BPE Training

Suppose our training corpus is the string: `l o w e s t l o w e r` (We insert spaces merely to show the current token boundaries).

Initialization: Base vocabulary: $\{\mathrm{l}, \mathrm{o}, \mathrm{w}, \mathrm{e}, \mathrm{s}, \mathrm{t}, \mathrm{r}\}$

Iteration 1: Frequencies of adjacent pairs:

- (l, o): 2

- (o, w): 2
- (w, e): 2
- (e, s): 1
- (s, t): 1
- (t, l): 1
- (e, r): 1

The pair (l, o) is tied for the most frequent. We merge it. New token: `lo`. Corpus becomes: `lo w e s t lo w e r`

Iteration 2: Frequencies of pairs:

- (`lo`, w): 2
- (w, e): 2
- (e, s): 1 ...

Merge (`lo`, w). New token: `low`. Corpus becomes: `low e s t low e r`

Iteration 3: Merge (w, e)? No, "w" is now part of "low". The pair (`low`, e) occurs twice. Merge it. New token: `lowe`. Corpus becomes: `lowe s t lowe r`

After 3 merges, our vocabulary is $\{\mathrm{l}, \mathrm{o}, \mathrm{w}, \mathrm{e}, \mathrm{s}, \mathrm{t}, \mathrm{r}, \mathrm{lo}, \mathrm{low}, \mathrm{lowe}\}$. The word "lowest" is tokenized as [`lowe`, s, t], and "lower" is [`lowe`, r]. The tokenizer learned the root morphological structure purely statistically!

8.6 Systems and Implementation Consequences

Tokenization is notoriously complex in engineering systems:

- **Pre-tokenization:** Before BPE, we typically use regex to split text into words and punctuation. If we don't, BPE might create tokens that span across word boundaries (e.g., merging the end of one word and the start of the next), which damages syntactic generalization.
- **Byte-Fallbacks:** To ensure the tokenizer can encode *any* string without "[UNK]" (unknown) tokens, modern tokenizers (like LLaMA's) operate on UTF-8 bytes rather than Unicode characters.
- **Embedding Matrix Size:** The dimension of the first and last weight matrices in the model is exactly $V \times d$. For $V = 100,000$ and $d = 4096$, the embedding matrix takes 1.6 GB of memory in fp32. This is often a significant portion of a smaller model's parameter count.

Neuroscience Analogy: Phonemes and Visual Features vs. Tokens

Where the analogy helps. Just as humans decompose spoken language into a finite set of phonemes (sound units) or written words into visual strokes/features, an LLM decomposes text into a finite set of subword tokens. In both cases, the complex continuous signal of language is chunked into discrete components that the processing system can handle.

Where the analogy breaks. Human chunking is profoundly influenced by semantics and morphology; we parse "unbelievable" into prefix, root, and suffix because of their meanings. BPE is purely statistical and frequency-driven. If a typo appears often enough in the training data, BPE will happily dedicate a dedicated token to it.

What intuition to keep. Tokens are the fundamental "atoms" of meaning for the model. However, these atoms are derived from compression algorithms, not linguistic theory. The model's "understanding" of a word is heavily dependent on how the tokenizer happened to slice it.

2026 Update: Byte-Level and Token-Free Models

While BPE is dominant, the field is actively researching ways to eliminate tokenizers entirely. Operating directly on raw bytes removes the complexity of the tokenizer pipeline and prevents issues where BPE behaves poorly on heavily numeric data (arithmetic), code spacing, or languages underrepresented in the tokenizer's training data.

Systems like **Megabyte** [**megabyte_2023**] and **BLT** [**blt_2024**] propose hierarchical architectures: a small recurrent or local module processes raw bytes and clusters them into "patch" representations, which are then processed by a global Transformer. This provides the flexibility of bytes without the massive sequence-length scaling penalty in the attention mechanism.

8.7 Common Misconceptions

1. **"One token equals one word."** On average in English, 1 token is about 0.75 words, or 4 characters. But common words ("the") are 1 token, while rare/complex words ("neuroscience") might be 2 or 3 tokens.

2. **"The model sees the characters inside a token."** It does not. If "cat" is tokenized as ID #2503, the model simply receives a vector mapping to #2503. It has no way of knowing that "c", "a", and "t" are the constituent letters unless it deduces it statistically from other tokens containing those letters. This is why LLMs struggle with character-level tasks like "how many r's in strawberry".

3. **"Tokenization is learned alongside the language model."** Tokenization is completely disjoint. The tokenizer is trained once on a corpus to build a fixed mapping dictionary.

The neural network is trained later using that fixed dictionary.

8.8 Exercises

Warmup

1. If your text corpus has $1,000,000$ characters and your vocabulary size is 1000, what is the absolute minimum and maximum number of tokens it could take to encode the corpus?

2. Why do LLMs typically learn a completely separate token for a capitalized word (e.g., "Apple") vs its lowercase version ("apple"), rather than sending a "capitalization" token?

Derivation

1. Let the vocabulary size V grow arbitrarily large. Trace the impact of this on (a) sequence length, (b) embedding memory usage, and (c) the probability of an OOV (out-of-vocabulary) word. What prevents us from setting $V = 1,000,000$?

Implementation

1. [**Prepares for CS336 A1**] Write a Python script to train a basic BPE tokenizer. Input: a list of strings (using characters as initial tokens). Output: a sequence of merge rules. Show the state of your vocabulary after 10 merges.

Open-Ended

1. Discuss how BPE tokenization impacts multi-lingual language models. If the training data corpus is 90% English and 10% Chinese, what will the resulting BPE vocabulary look like, and how will this affect the efficiency of inferencing Chinese text?

8.9 Further Reading

- "Sennrich" [4]: Neural Machine Translation of Rare Words with Subword Units. The paper that introduced BPE to NLP.

- **blt_2024**: BLT: Byte Latent Transformer. A cutting-edge architecture exploring how to replace subword tokenization entirely with byte-level latent processing.

- **byt5_2021**: ByT5: Towards a token-free future with pre-trained byte-to-byte models.

Chapter 9

From N-Grams to Neural Language Models

Key Insight

After reading this chapter, you will be able to:

1. Construct an n-gram language model and explain its fundamental limitations (the curse of dimensionality).
2. Describe how continuous vector embeddings solve the data sparsity problem.
3. Formulate a simple Neural Probabilistic Language Model (NPLM).
4. Understand why recurrent neural networks (RNNs) failed to scale for long context lengths.

9.1 Big Picture

Before Transformers dominated NLP, the field progressed through two distinct epochs: the n-gram era and the early neural era. This chapter covers both, not just for historical context, but because understanding *why* these older methods failed provides the precise motivation for the mathematical structure of the Transformer.

The n-gram method represents language as exact counts of discrete sequences. It is simple and fast, but it suffers from the "curse of dimensionality"—it cannot generalize to sequences it has never seen before. The first neural language models fixed this by embedding discrete tokens into a continuous vector space, allowing them to generalize based on geometric similarity. However, their architectures (feed-forward or recurrent) struggled to efficiently route information across long distances. This sets the stage appropriately for Chapter 9, where we introduce the attention mechanism.

9.2 Formal Setup and Notation

Recall the language modeling problem from Chapter 6: we want to estimate $P(x_t \mid x_{<t})$. Let $C(w_1, \ldots, w_k)$ denote the number of times the exact sequence of tokens $(w_1, \ldots, w_k)$ appears in our training corpus.

We map each token $x_i \in \mathcal{V}$ to an integer index $v \in \{1, \ldots, V\}$. A **one-hot vector** $\mathbf{e}_v \in \{0, 1\}^V$ is a vector where the v-th element is 1 and all other elements are 0.

An **embedding matrix** $\mathbf{W}_E \in \mathbb{R}^{V \times d}$ maps tokens to continuous vectors in d-dimensional space. The embedding for token v is exactly the v-th row of $\mathbf{W}_E$, which can be mathematically written as $\mathbf{x} = \mathbf{e}_v^\mathsf{T} \mathbf{W}_E \in \mathbb{R}^d$.

9.3 The N-Gram Language Model

An n-gram model makes a simplifying Markov assumption: the probability of the next token depends *only* on the preceding $n-1$ tokens.

$$P(x_t \mid x_1, \ldots, x_{t-1}) \approx P(x_t \mid x_{t-n+1}, \ldots, x_{t-1}). \tag{9.1}$$

To estimate this probability, we use maximum likelihood estimation based on counts:

$$P_{\text{MLE}}(x_t = v \mid x_{t-n+1}, \ldots, x_{t-1}) = \frac{C(x_{t-n+1}, \ldots, x_{t-1}, v)}{C(x_{t-n+1}, \ldots, x_{t-1})}. \tag{9.2}$$

9.3.1 The Curse of Dimensionality

Suppose we build a 5-gram model over a vocabulary of $V = 50,000$. There are $V^4 = 6.25 \times 10^{18}$ possible histories of length 4. Even a massive corpus like Common Crawl (trillions of tokens) will not contain the vast majority of these histories.

If we encounter a history h that never appeared in the training set, $C(h) = 0$, and the probability is undefined. If we encounter a history that appeared, but a valid next word never followed it in the training set, $P_{\text{MLE}} = 0$, resulting in infinite cross-entropy (-log $0 = \infty$).

To fix this, n-gram models use **smoothing** (e.g., Kneser-Ney), which falls back to lower-order models (4-grams, then 3-grams) when data is sparse. But smoothing does not solve the fundamental flaw: n-gram models treat tokens as atomic symbols. They do not know that "cat" and "dog" are similar. If a model learns that "A dog is sleeping in the _____" predicts "bed", it cannot generalize this to "A cat is sleeping in the _____" unless that exact 6-gram also appeared.

9.4 Worked Example: Trigram Calculation

Suppose our corpus is exactly the following three sentences:

- `<s> i love deep learning </s>`
- `<s> i love machine learning </s>`
- `<s> cats love deep sleeping </s>`

We want to build a trigram ($n = 3$) model and compute the probability of "`learning`" given the history "`love deep`".

1. Count the history: $C(\texttt{love}, \texttt{deep}) = 2$. 2. Count the full trigram: $C(\texttt{love}, \texttt{deep}, \texttt{learning}) = 1$. 3. Compute MLE:

$$P_{\text{MLE}}(\texttt{learning} \mid \texttt{love}, \texttt{deep}) = \frac{1}{2} = 0.5. \tag{9.3}$$

Now, compute the probability of "`learning`" given "`love machine`": 1. $C(\texttt{love}, \texttt{machine}) = 1$. 2. $C(\texttt{love}, \texttt{machine}, \texttt{learning}) = 1$. 3. $P_{\text{MLE}} = \frac{1}{1} = 1.0$.

9.5 The Neural Probabilistic Language Model (NPLM)

In 2003, Bengio et al. proposed a solution to the curse of dimensionality: represent each token as a continuous vector. If the vectors for "cat" and "dog" are close in the d-dimensional space, the neural network's smooth prediction function will automatically assign similar probabilities to contexts involving them.

A classic window-based neural language model computes $P(x_t \mid x_{t-n+1}, \ldots, x_{t-1})$ via three steps:

1. **Embed**: Look up the vector for each of the $n - 1$ context tokens.
2. **Concatenate**: Flatten these vectors into a single vector $\mathbf{h}_{\text{in}} \in \mathbb{R}^{(n-1)d}$.
3. **Project**: Pass $\mathbf{h}_{\text{in}}$ through a multi-layer perceptron (MLP) to get a hidden state, then project it to the vocabulary size to get **logits** $\mathbf{z} \in \mathbb{R}^V$.
4. **Softmax**: Convert logits to probabilities.

$$P(x_t = v \mid \text{context}) = \text{softmax}(\mathbf{z})_v = \frac{\exp(z_v)}{\sum_{k=1}^{V} \exp(z_k)}. \tag{9.4}$$

9.6 The Recurrent Era and Its Failure Modes

Window-based feed-forward neural networks still suffer from a fixed context window n. To allow infinite context, researchers turned to Recurrent Neural Networks (RNNs) and their variants (LSTMs, GRUs). In an RNN, the hidden state at time t depends on the input at time t and the hidden state at time $t - 1$:

$$\mathbf{h}_t = f(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}). \tag{9.5}$$

Why did RNNs lose to Transformers?

1. **Vanishing Gradients**: Gradients must flow backwards through time (BPTT). Over hundreds of steps, the repeated multiplication by the recurrent weight matrix $\mathbf{W}_h$ causes gradients to vanish or explode, making it impossible to learn long-term dependencies.

2. **State Bottleneck**: The entire history of the sequence must be compressed into a single, fixed-size vector $\mathbf{h}_t$. If the sequence is 10,000 tokens long, $\mathbf{h}_t$ struggles to remember the first 100 tokens.

3. **Lack of Parallelism**: To compute $\mathbf{h}_t$, you strictly must compute $\mathbf{h}_{t-1}$ first. This serial dependency makes RNNs fundamentally hostile to GPU hardware, which excels at doing thousands of operations simultaneously. During training, we know the whole sequence, but an RNN still forces us to process it sequentially.

These three flaws—vanishing gradients, state bottlenecks, and serial execution—are exactly the problems the Transformer's attention mechanism solves.

9.7 Systems and Implementation Consequences

The transition from n-grams to neural models shifted NLP from memory-bound table lookups to compute-bound dense linear algebra.

- **Embedding tables** are sparse lookups. We do not actually multiply $\mathbf{e}_v^\top \mathbf{W}_E$ (which is mostly zeros), we simply index into the v-th row in PyTorch using 'nn.Embedding'.

- **The Softmax Bottleneck**: The final projection matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$ is massive. If $d = 4096$ and $V = 100,000$, computing the logits takes a huge number of FLOPs. The softmax operation itself requires calculating the sum of exponentials across the entire vocabulary V for every batch item and token position. Early NPLMs used approximations like hierarchical softmax or negative sampling to avoid computing the full denominator, but modern hardware is fast enough that we simply compute the full softmax.

Neuroscience Analogy: Population Coding and Embeddings

Where the analogy helps. How does the brain represent the concept of "apple"? It is not a single "apple neuron" firing (a one-hot encoding). Instead, it is a pattern of activation across millions of neurons (a distributed representation). If you see a pear, the neural pattern will be highly overlapping with the apple pattern. This is precisely how continuous vector embeddings work in neural networks: similar concepts map to vectors with high cosine similarity.

Where the analogy breaks. Neural networks use dense float vectors where every dimension participates in every concept. The brain uses sparse coding—only a small, specialized subset of neurons fires for any given stimulus, which is vastly more energy efficient.

What intuition to keep. Moving from distinct n-gram symbols to continuous vectors enables generalization. A model learns about a new word simply by mapping its vector near the vectors of similar words it already knows.

9.8 Common Misconceptions

1. **"Embeddings are fixed mappings from words to meaning."** While Word2Vec embeddings were static, modern language models compute contextual embeddings. The vector for "bank" in "river bank" is entirely different from "bank" in "bank account" in the deeper layers of a modern model, though the initial lookup vector is the same.

2. **"RNNs failed because they couldn't model language."** RNNs and LSTMs were highly successful language models. They failed because they could not scale on GPU hardware. The Transformer won not because it has a better linguistic inductive bias, but because it is massively parallelizable, allowing us to train on exponentially more data.

9.9 Exercises

Warmup

1. In a 4-gram model over a vocabulary of 10,000, how many possible 4-grams exist?

2. If an embedding matrix is $100,000 \times 4096$ using 16-bit floats (2 bytes per number), how much memory does it consume?

Derivation

1. Prove mathematically why repeatedly multiplying a vector by a matrix $\mathbf{W}$ (as in a simplified RNN $\mathbf{h}_t = \mathbf{W}\mathbf{h}_{t-1}$) whose largest eigenvalue is less than 1 leads to exponentially decaying

gradients (the vanishing gradient problem).

Implementation

1. [**Prepares for CS336 A1**] Write a PyTorch class ‘SimpleNPLM‘ that inherits from ‘nn.Module‘. It should take a sequence of $(n-1)$ integer tokens, map them through an ‘nn.Embedding‘, concatenate them, pass them through a linear layer and a ReLU, and finally output logits over the vocabulary.

Open-Ended

1. If RNNs suffered from a state bottleneck (compressing all history into one vector), how do you think modern architectures like the Transformer solve this? (Preview your intuition before Chapter 9).

9.10 Further Reading

- “A Neural Probabilistic Language Model” [2]: A Neural Probabilistic Language Model. Introduces word embeddings and the neural approach to curing the curse of dimensionality.
- **mikolov2013**: Efficient Estimation of Word Representations in Vector Space (Word2Vec). A milestone paper explaining the geometric properties of word embeddings.

Part III

Transformer Internals

Chapter 10

Attention Mechanisms

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

10.1 Big Picture

TODO: Situate this chapter in the overall narrative.

10.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

10.3 Main Ideas

TODO: Core technical content.

10.4 Worked Example

TODO: Complete worked example.

10.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

10.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

10.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

10.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 11

Transformer Blocks from Scratch

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

11.1 Big Picture

TODO: Situate this chapter in the overall narrative.

11.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

11.3 Main Ideas

TODO: Core technical content.

11.4 Worked Example

TODO: Complete worked example.

11.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

11.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

11.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

11.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 12

Norms, Residuals, and Positional Encodings

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

12.1 Big Picture

TODO: Situate this chapter in the overall narrative.

12.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

12.3 Main Ideas

TODO: Core technical content.

12.4 Worked Example

TODO: Complete worked example.

12.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

12.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

12.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

12.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 13

Hyperparameters, Initialization, and Optimization

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

13.1 Big Picture

TODO: Situate this chapter in the overall narrative.

13.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

13.3 Main Ideas

TODO: Core technical content.

13.4 Worked Example

TODO: Complete worked example.

13.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

13.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

13.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

13.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 14

Architectural Variants and Extensions

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

14.1 Big Picture

TODO: Situate this chapter in the overall narrative.

14.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

14.3 Main Ideas

TODO: Core technical content.

14.4 Worked Example

TODO: Complete worked example.

14.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

14.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

14.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

14.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part IV

Systems and Implementation

Chapter 15

Resource Accounting

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

15.1 Big Picture

TODO: Situate this chapter in the overall narrative.

15.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

15.3 Main Ideas

TODO: Core technical content.

15.4 Worked Example

TODO: Complete worked example.

15.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

15.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

15.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

15.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 16

GPUs, TPUs, and Memory Hierarchy

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

16.1 Big Picture

TODO: Situate this chapter in the overall narrative.

16.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

16.3 Main Ideas

TODO: Core technical content.

16.4 Worked Example

TODO: Complete worked example.

16.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

16.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

16.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

16.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 17

Kernels, CUDA, Triton, and FlashAttention

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

17.1 Big Picture

TODO: Situate this chapter in the overall narrative.

17.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

17.3 Main Ideas

TODO: Core technical content.

17.4 Worked Example

TODO: Complete worked example.

17.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

17.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

17.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

17.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 18

Parallelism and Distributed Training

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

18.1 Big Picture

TODO: Situate this chapter in the overall narrative.

18.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

18.3 Main Ideas

TODO: Core technical content.

18.4 Worked Example

TODO: Complete worked example.

18.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

18.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

18.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

18.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 19

Inference and Serving

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

19.1 Big Picture

TODO: Situate this chapter in the overall narrative.

19.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

19.3 Main Ideas

TODO: Core technical content.

19.4 Worked Example

TODO: Complete worked example.

19.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

19.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

19.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

19.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part V

Scaling, Evaluation, and Recipes

Chapter 20

Scaling Laws

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

20.1 Big Picture

TODO: Situate this chapter in the overall narrative.

20.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

20.3 Main Ideas

TODO: Core technical content.

20.4 Worked Example

TODO: Complete worked example.

20.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

20.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

20.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

20.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 21

Model Recipes and Open-Model Case Studies

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

21.1 Big Picture

TODO: Situate this chapter in the overall narrative.

21.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

21.3 Main Ideas

TODO: Core technical content.

21.4 Worked Example

TODO: Complete worked example.

21.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

21.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

21.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

21.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 22

Evaluation

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

22.1 Big Picture

TODO: Situate this chapter in the overall narrative.

22.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

22.3 Main Ideas

TODO: Core technical content.

22.4 Worked Example

TODO: Complete worked example.

22.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

22.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

22.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

22.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part VI

Data

Chapter 23

Data Sources, Crawling, and Cleaning

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

23.1 Big Picture

TODO: Situate this chapter in the overall narrative.

23.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

23.3 Main Ideas

TODO: Core technical content.

23.4 Worked Example

TODO: Complete worked example.

23.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

23.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

23.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

23.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 24

Filtering, Deduplication, and Data Mixing

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

24.1 Big Picture

TODO: Situate this chapter in the overall narrative.

24.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

24.3 Main Ideas

TODO: Core technical content.

24.4 Worked Example

TODO: Complete worked example.

24.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

24.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

24.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

24.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Part VII

Alignment and Reasoning

Chapter 25

Instruction Tuning and Preference Tuning

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

25.1 Big Picture

TODO: Situate this chapter in the overall narrative.

25.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

25.3 Main Ideas

TODO: Core technical content.

25.4 Worked Example

TODO: Complete worked example.

25.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

25.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

25.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

25.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 26

RLHF, DPO, GRPO, and Reasoning RL

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

26.1 Big Picture

TODO: Situate this chapter in the overall narrative.

26.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

26.3 Main Ideas

TODO: Core technical content.

26.4 Worked Example

TODO: Complete worked example.

26.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

26.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

26.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

26.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Chapter 27

Safety, Reward Hacking, and Deployment

Key Insight

After reading this chapter, you will be able to:

1. TODO: Learning goal 1
2. TODO: Learning goal 2
3. TODO: Learning goal 3

27.1 Big Picture

TODO: Situate this chapter in the overall narrative.

27.2 Formal Setup and Notation

TODO: Define notation and state assumptions.

27.3 Main Ideas

TODO: Core technical content.

27.4 Worked Example

TODO: Complete worked example.

27.5 Systems and Implementation Consequences

TODO: Implementation and performance implications.

Neuroscience Analogy: TODO: Title

Where the analogy helps. TODO
Where the analogy breaks. TODO
What intuition to keep. TODO

27.6 Common Misconceptions

1. **Misconception 1.** TODO: State and correct.
2. **Misconception 2.** TODO: State and correct.
3. **Misconception 3.** TODO: State and correct.

27.7 Exercises

Warmup

1. TODO

Derivation

1. TODO

Implementation

1. TODO

Open-Ended

1. TODO

27.8 Further Reading

- TODO: Annotated reference 1
- TODO: Annotated reference 2
- TODO: Annotated reference 3

Appendices

Appendix A

Proof Techniques and Derivations

TODO: Content for this appendix.

Appendix B

CUDA and Triton Cookbook

TODO: Content for this appendix.

Appendix C

Linguistics Quick Reference

TODO: Content for this appendix.

Appendix D

Neuroscience Analogy Index

TODO: Content for this appendix.

Appendix E

Reproducibility and Debugging

TODO: Content for this appendix.

References

[1] "Prediction and Entropy of Printed English". In: (1950). URL: https://www.princeton.edu/~wbialek/rome/refs/shannon_51.pdf.

[2] "A Neural Probabilistic Language Model". In: (2003). Used a feedforward neural network over last n words to predict the next word in a sequence. URL: https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf.

[3] OpenAI. "Language Models are Unsupervised Multitask Learners". In: (2019). 1.5B parameters | Pioneered stage release. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[4] "Sennrich". In: (). arXiv: 1508.07909. URL: https://arxiv.org/abs/1508.07909.